

Detailed Web GIS Code Tutorial & Reference Guide

A Complete Section-by-Section Review with Exhaustive In-Line Documentation and Architecture Analysis

1. Overview of Dashboard Enhancements & Hidden Leaflet Bugs

Building open-source Web GIS systems requires careful management of data updates and rendering boundaries. In typical static HTML apps, vector graphics (like polygon coordinates loaded from our Lille parks dataset) are styled directly on the base body canvas layer using target CSS properties. In multi-tier interactive dashboards, this results in a critical bug that causes your textures to break.

When you run filter calculations, use drop-downs, or add buffer rings, Leaflet handles the changes by invoking `clearLayers()` and then `addData()`. This wipes the existing paths clean and updates the map elements. However, during this process, **Leaflet completely regenerates its own internal dynamic SVG root element container**.

If you rely on a static `<svg><defs>` asset block written at the top of your HTML markup, Leaflet's internal shapes will completely lose track of it when the layer reloads, causing all your custom hatch patterns to vanish.

The Dynamic Solution: We fixed this permanently by removing the static HTML blocks and using a smart JavaScript function called `injectSvgPatterns()`. This script hooks directly into Leaflet's dynamic container canvas pane (`overlayPane`). Every single time data is loaded, filtered, or a buffer ring is drawn, the function fires automatically, injecting your custom stripes right where Leaflet can find them.

2. In-Depth Section Breakdowns

Section 1: Framework Libraries & CDN Ingestion

The document head loads everything needed for client-side processing: Bootstrap builds our column layout; Leaflet handles map zooming, panning, and rendering layers; Leaflet-AJAX fetches data files asynchronously in the background; and Turf.js acts as a mini-desktop GIS processing engine right inside your web browser.

Section 2: Layout Geometry & Absolute Viewport Overlays

The user interface uses Bootstrap's grid system to divide the screen into a sidebar list (`col-md-3`) and a map element pane (`col-md-9`). To add advanced features without cluttering the sidebar, three components are positioned using absolute coordinates: a sliding layer visibility toggle checklist, an interactive graphic map legend table matching your French dataset labels, and a traditional cartographic North Arrow vector.

Section 3: Map Viewport Engine Initialization

The Leaflet instance starts up by adding standard mapping features, such as metric scale bars and fullscreen view modes. It also includes an active movement listener: `mymap.on('move', ...)` monitors the center point of the camera view and formats it as crisp decimal numbers, updating the latitude and longitude labels in the footer container instantly.

Section 4: Object-Context Asynchronous Data Loop

The `L.GeoJSON.AJAX` loader parses incoming data files row by row. To avoid classic variable overwriting bugs, we build the popup boxes and sidebar item modules as active jQuery elements. This lets us bind click listeners directly to the specific layer objects using `layer.getBounds().getCenter()`. This ensures that clicking any sidebar item centers the camera accurately on that exact park polygon without getting confused by other shapes.

Section 5: Two-Way Synchronized Interface Filtering Loop

When you search or filter for a park type using the selector drop-down, the dashboard updates two areas simultaneously. First, it filters the sidebar text entry boxes using jQuery `.show()` and `.hide()` loops. Second, it alters the map itself: it purges all shapes using `clearLayers()`, extracts matching features from a memory copy using array evaluations, and pushes them back onto the map using `addData()`. Finally, it fires the pattern injector again to re-apply your custom fills cleanly.

Section 6: Turf.js Proximity Buffer Generator

The buffering engine converts user text input into a true numeric parameter via `parseFloat()`. It validates the input number, strips any old shapes off the map using `mymap.removeLayer()`, reads the current geographic footprint coordinates, and runs a Turf buffer calculation. It then draws a translucent overlay on your screen and smoothly shifts the camera focus using `mymap.fitBounds()`.

3. The Fully Annotated Source Deployment Code

Below is your complete, production-ready `index.html` code block, featuring comprehensive inline comments detailing how every section functions.

```

<!doctype html>
<html lang="en">

<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>WebMap 101 - Lille Dashboard</title>

  <!-- =====
    CORE EXTERNAL LIBRARIES AND ASSETS CDNs
    ===== -->

  <!-- Bootstrap: Handles responsive grids and element alignment styling -->
  <link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/css/
bootstrap.min.css">
  <!-- Leaflet: Provides base web map layout rendering logic -->
  <link rel="stylesheet" href="https://unpkg.com/leaflet@1.0.1/dist/leaflet.css" />
  <!-- Fullscreen Plugin: Supplies stylesheets for full screen expansion mode -->
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/leaflet.fullscreen/
1.0.2/Control.FullScreen.css" />

  <!-- Leaflet Mapping Base Framework Script Engine -->
  <script src="https://unpkg.com/leaflet@1.0.1/dist/leaflet.js"></script>
  <!-- Fullscreen Expansion View Extension Script Module -->
  <script src="https://cdnjs.cloudflare.com/ajax/libs/leaflet.fullscreen/1.0.2/
Control.FullScreen.js"></script>

  <!-- jQuery: Simplifies DOM element selection, structural insertion, and animation -->
  <script src="https://code.jquery.com/jquery-2.2.4.min.js"></script>
  <!-- Leaflet AJAX Extension: Handles background data streaming without freezing the UI -->
  <script src="resources/leaflet-ajax-gh-pages/dist/leaflet.ajax.js"></script>
  <!-- Turf.JS: Performs desktop-grade client-side spatial and vector processing loops -->
  <script src="https://cdn.jsdelivr.net/npm/@turf/turf@6/turf.min.js"></script>
  <!-- GeometryUtil Plugin: Computes snap points, linear bearings, and geometric distances -->
  <script src="https://cdn.jsdelivr.net/npm/leaflet-geometryutil@0.9.3/src/
leaflet.geometryutil.min.js"></script>

<style>
  /* =====
    LAYOUT GEOMETRY & VIEWPORT DESIGN STYLES
    ===== */
  #header { height: 75px; background-color: rgb(15, 121, 156); color: white; }
  #mapdiv { height: 850px; background-color: #e5e9f0; }
  /* Scrollable Sidebar Panel: Houses list elements when they leak past screen boundaries
*/
  #side_panel { height: 850px; background-color: rgb(174, 188, 223); overflow-y: auto;
padding-bottom: 20px; }
  #footer { height: 75px; background-color: darkgrey; }

  /* Floating Absolute Overlays: Placed cleanly above map layers via absolute positioning
*/
  .toc-panel {
    position: absolute; top: 10px; right: 50px; z-index: 1000;
    background: white; padding: 10px; border-radius: 4px;

```

```

        box-shadow: 0 1px 5px rgba(0,0,0,0.4); max-width: 250px; font-family: Arial, sans-
        serif;
    }
    .toc-header { font-weight: bold; cursor: pointer; padding-bottom: 5px; border-bottom:
1px solid #ccc; user-select: none; }
    .toc-content { margin-top: 8px; }

    /* Graphic Map Legend Stacking Order Rules Layout */
    .map-legend {
        position: absolute; bottom: 30px; right: 10px; z-index: 999;
        background: rgba(255, 255, 255, 0.9); padding: 10px; border-radius: 5px;
        box-shadow: 0 0 15px rgba(0,0,0,0.2); font-family: Arial, sans-serif; font-size:
12px; line-height: 18px;
    }
    .legend-color-box { width: 18px; height: 18px; float: left; margin-right: 8px; opacity:
0.7; border: 1px solid #555; }

    /* Compass North Orientation Icon Frame Element Wrapper */
    .north-arrow-control {
        position: absolute; top: 90px; left: 10px; z-index: 1000;
        background: white; padding: 5px; border-radius: 4px;
        box-shadow: 0 1px 5px rgba(0,0,0,0.4); width: 34px; height: 34px; text-align:
center;
    }
</style>
</head>

<body>
    <!-- Header Container Area -->
    <div id="header" class="col-md-12">
        <h1 class="text-center" style="margin-top: 15px; font-weight: bold;">Lille WebGIS</h1>
    </div>

    <!-- Control Sidebar Area -->
    <div id="side_panel" class="col-md-3">
        <h1 class="text-center">Attractions</h1>
        <hr>

        <!-- Dynamic Proximity Analytical Input Interface Panel -->
        <h4 class="text-center" style="text-decoration: underline;">Actions Rapides</h4>
        <div class="form-group" style="padding: 5px 0;">
            <label for="buffer-input">Distance du Buffer (mètres):</label>
            <input type="number" id="buffer-input" class="form-control" value="100" min="1"
max="5000">
        </div>
        <button id="btnBuffer" class="form-control btn btn-warning">Générer le Buffer</button>
        <button id="ZoomToCitadelle" class="form-control btn btn-primary" style="margin-top:
5px;">Zoomer sur la Citadelle</button>
        <hr>

        <!-- Multi-Tier Selection Filtering Interface Dropdown Input -->
        <div style="padding: 10px 0;">
            <label for="park-search">Filtrer les zones vertes par type :</label>

```

```

        <input list="park-types" id="park-search" class="form-control"
placeholder="Rechercher un type...">
        <datalist id="park-types">
            <option value="Toutes les zones vertes">Toutes les zones vertes</option>
            <option value="Cimetière">Cimetière</option>
            <option value="Espace vert public">Espace vert public</option>
            <option value="Ferme pédagogique">Ferme pédagogique</option>
            <option value="Jardin d'Habitants">Jardin d'Habitants</option>
        </datalist>
    </div>
    <hr>
    <!-- Target placement slot where our parsed data items will land -->
    <div id="park-list-container"></div>
</div>

<!-- Mapping Canvas Container Area -->
<div id="mapdiv" class="col-md-9" style="position: relative;">
    <!-- Table of Contents Layer Toggles Checklist Widget View -->
    <div class="toc-panel">
        <div class="toc-header" onclick="toggleTOC()">Couches Cartographiques <span id="toc-
toggle-icon" style="float:right;">▼</span></div>
        <div class="toc-content" id="toc-items">
            <div class="checkbox"><label><input type="checkbox" id="layer-osm" checked>
Carte de base (OSM)</label></div>
            <div class="checkbox"><label><input type="checkbox" id="layer-parks" checked>
Espaces Verts de Lille</label></div>
            <div class="checkbox"><label><input type="checkbox" id="layer-citadelle"
checked> Citadelle de Lille (Point)</label></div>
        </div>
    </div>

    <!-- Graphic Map Legend Block Component View -->
    <div class="map-legend">
        <h4><b>Légende</b></h4>
        <div><div class="legend-color-box" style="background:#7f8c8d; background-image:
repeating-linear-gradient(45deg, transparent, transparent 2px, #fff 2px, #fff 4px);"></
div>Cimetière</div>
        <div><div class="legend-color-box" style="background:#2ecc71; background-image:
repeating-linear-gradient(-45deg, transparent, transparent 2px, #fff 2px, #fff 4px);"></
div>Espace vert public</div>
        <div><div class="legend-color-box" style="background:#e67e22; background-image:
repeating-linear-gradient(90deg, transparent, transparent 2px, #fff 2px, #fff 4px);"></div>Ferme
pédagogique</div>
        <div><div class="legend-color-box" style="background:#3498db; background-image:
repeating-linear-gradient(0deg, transparent, transparent 2px, #fff 2px, #fff 4px);"></div>Jardin
d'Habitants</div>
        <div><div class="legend-color-box" style="background:#95a5a6; "></div>Autre / Non
classé</div>
    </div>

    <!-- Cartesian Compass Indicator Graphic Asset Node -->
    <div class="north-arrow-control">
        <span style="font-weight:bold; font-size:16px; color:#c0392b; display:block; margin-

```

```

top:-2px;">▲</span>
    <span style="font-size:10px; font-weight:bold; display:block; margin-top:-4px;">N</
span>

</div>

<!-- Bottom Coordinates Readout Footer Area -->
<div id="footer" class="col-md-12">
    <h4 id="map_coords" class="text-center" style="margin-top: 15px;">Latitude: 50.6333
Longitude: 3.06667 Zoom Level: 14</h4>
</div>

<script>
    // -----
    // MAIN APPLICATION ENGINE SCRIPTS INITIALIZATION
    // -----

    // Instantiate Map pane framework and attach native plugin fullscreen hooks
    var mymap = L.map('mapdiv', {
        fullscreenControl: true,
        fullscreenControlOptions: { position: 'topleft' }
    });
    mymap.setView([50.6333, 3.06667], 14);

    // Append international OpenStreetMap base background layout tiles
    var backgroundLayer = L.tileLayer('http://{s}.tile.osm.org/{z}/{x}/{y}.png').addTo(mymap);

    // Append standard cartographic scale system bar (Metric standard strictly forced)
    L.control.scale({ position: 'bottomleft', imperial: false }).addTo(mymap);

    // Append focal marker representation point matching our Lille Citadelle coordinates
    var citadellemarker = L.marker([50.6409, 3.0446]).addTo(mymap);
    citadellemarker.bindPopup("<h3><b>Citadelle de Lille</b></h3><hr><p>Forteresse
pentagonale built by Vauban in 1667.</p>");

    // Action button hook to smoothly center focus back over the point feature coordinate
    $("#ZoomToCitadelle").on('click', function () { mymap.setView([50.6409, 3.0446],
17); });

    // Viewport camera motion tracker: prints exact string transformations on the dashboard
    footer
    mymap.on('move', function () {
        var center = mymap.getCenter();
        var zoom = mymap.getZoom();
        $("#map_coords").html("Latitude: " + center.lat.toFixed(5) + " Longitude: " +
center.lng.toFixed(5) + " Zoom Level: " + zoom);
    });

    // Sliding Animation Toggle Panel for managing custom layer visibilities
    function toggleTOC() {
        var items = $("#toc-items");
        var icon = $("#toc-toggle-icon");

```

```

        items.is(":visible") ? (items.slideUp(), icon.text("▲")) : (items.slideDown(),
icon.text("▼"));
    }

    // Layer Mount/Unmount Form Inputs Handlers Mapping
    $("#layer-osm").on('change', function() { this.checked ?
mymap.addLayer(backgroundLayer) : mymap.removeLayer(backgroundLayer); });
    $("#layer-citadelle").on('change', function() { this.checked ?
mymap.addLayer(citadellemarker) : mymap.removeLayer(citadellemarker); });
    $("#layer-parks").on('change', function() { this.checked ?
mymap.addLayer(geojsonLayer) : mymap.removeLayer(geojsonLayer); });

    // Unique value mapping handler: parses target properties out to exact hex colors
    function getParkColor(typeValue) {
        if (!typeValue) return '#95a5a6';
        switch (typeValue.trim()) {
            case 'Cimetière': return '#7f8c8d';
            case 'Espace vert public': return '#2ecc71';
            case 'Ferme pédagogique': return '#e67e22';
            case 'Jardin d'Habitants': return '#3498db';
            default: return '#95a5a6';
        }
    }

    // Unique value mapping handler: maps asset tags out to our active structural patterns
ids
    function getParkPattern(typeValue) {
        if (!typeValue) return '';
        switch (typeValue.trim()) {
            case 'Cimetière': return 'url(#pattern-cemetery)';
            case 'Espace vert public': return 'url(#pattern-green)';
            case 'Ferme pédagogique': return 'url(#pattern-farm)';
            case 'Jardin d'Habitants': return 'url(#pattern-jardin)';
            default: return '';
        }
    }

    // CORE FIX: Injects repeating vector tile defs straight inside Leaflet's own active map
canvas namespace wrapper
    function injectSvgPatterns() {
        var svgCanvas = mymap.getPanes().overlayPane.querySelector('svg');
        if (!svgCanvas) return; // Safety check block in case vector assets have not fully
hit screen coordinates yet

        var defs = svgCanvas.querySelector('defs');
        if (!defs) {
            defs = document.createElementNS('http://www.w3.org/2000/svg', 'defs');
            svgCanvas.insertBefore(defs, svgCanvas.firstChild);
        }

        // Build cross hatched vector assets into Leaflet's dynamic coordinate engine space
        defs.innerHTML = `
            <pattern id="pattern-cemetery" width="10" height="10"

```



```

patternTransform="rotate(45)" patternUnits="userSpaceOnUse">
    <line x1="0" y1="0" x2="0" y2="10" stroke="#7f8c8d" stroke-width="3" />
</pattern>
<pattern id="pattern-green" width="12" height="12"
patternTransform="rotate(135)" patternUnits="userSpaceOnUse">
    <line x1="0" y1="0" x2="0" y2="12" stroke="#27ae60" stroke-width="3" />
</pattern>
<pattern id="pattern-farm" width="20" height="20" patternUnits="userSpaceOnUse">
    <line x1="0" y1="0" x2="0" y2="20" stroke="#d35400" stroke-width="4" />
</pattern>
<pattern id="pattern-jardin" width="8" height="8" patternTransform="rotate(90)"
patternUnits="userSpaceOnUse">
    <line x1="0" y1="0" x2="0" y2="8" stroke="#2980b9" stroke-width="2" />
</pattern>
`
};

}

var rawBackupGeoJSON = null; // Caches global original dataset clone to safely support
hard re-filter injections

// Initialize background data stream loader pipeline
var geojsonLayer = new L.GeoJSON.AJAX('parksLille.geojson', {
    style: function (feature) {
        var typeAttr = feature.properties.types;
        return {
            color: getParkColor(typeAttr),
            fillColor: getParkPattern(typeAttr), // Map reference pointing directly to
the injected SVG defs id
            weight: 3,
            opacity: 0.9,
            fillOpacity: 0.6
        };
    },
    onEachFeature: function (feature, layer) {
        // Compile textual metadata body strings safely
        var infoHtml = "<h4>" + (feature.properties.nom || "Espace Vert") + "</h4><hr>";
        if (feature.properties.quartier) infoHtml += "<b>Quartier:</b> " +
feature.properties.quartier + "<br>";
        if (feature.properties.types) infoHtml += "<b>Type:</b> " +
feature.properties.types + "<br>";
        if (feature.properties["surface"]) infoHtml += "<b>Surface:</b> " +
feature.properties["surface"].toFixed(0) + " m²<br>";
        if (feature.properties["Ouverture au public"]) infoHtml += "<b>Ouverture au
public:</b> " + feature.properties["Ouverture au public"] + "<br>";

        // Create Map Popup zoom button as an active object context vector
        var popupBtn = $("<button class='btn btn-success btn-xs btn-block'
style='margin-top: 10px;'>🔍 Zoomer sur cet espace</button>");
        popupBtn.on('click', function () {
            if (typeof layer.getBounds === 'function') {
                mymap.setView(layer.getBounds().getCenter(), 17);
            }
        });
    });

```

```

        var popupContainer = $("

</div>").append(infoHtml).append(popupBtn);
        layer.bindPopup(popupContainer[0]);

        // Create Sidebar element entry panel as a structural active context block
        var typeAttr = (feature.properties.types || "").trim();
        var itemContainer = $("

Lille Web GIS Project • Deep Dive Code Walkthrough & Comments



Page 10


```

```

screen area
    if (!rawBackupGeoJSON) return;
    geojsonLayer.clearLayers(); // Wipe out old lines

    if (filterKey === "Toutes les zones vertes" || selectedValue === "") {
        geojsonLayer.addData(rawBackupGeoJSON); // Restore complete dataset copy
    } else {
        // Filter matching features from background memory array list
        var filteredFeatures = rawBackupGeoJSON.features.filter(function(feat) {
            var fType = (feat.properties.types || "").trim();
            return fType === filterKey || fType === selectedValue;
        });
        var filteredGeoJSONCollection = { type: "FeatureCollection", features:
filteredFeatures };
        geojsonLayer.addData(filteredGeoJSONCollection); // Append filtered features
chunk cleanly
    }

    // CRITICAL STEP: Always call the texture generator function to patch canvas
patterns after map layers update
    injectSvgPatterns();
});

// Client-Side Spatial Proximity Suite Analytics Control Block
var bufferLayer = null;

$("#btnBuffer").on('click', function () {
    // Remove previous geometry overlay chunk if already active on map canvas
    if (bufferLayer) {
        mymap.removeLayer(bufferLayer);
        bufferLayer = null;
    }

    if ($(this).text() === "Générer le Buffer") {
        // Read live user choices value text fields and cast smoothly to floats
        var distanceVal = parseFloat($("#buffer-input").val());

        if (isNaN(distanceVal) || distanceVal <= 0) {
            alert("Veuillez entrer une distance de buffer valide (supérieure à 0).");
            return;
        }

        var originalGeoJSON = geojsonLayer.toGeoJSON();
        if (!originalGeoJSON || !originalGeoJSON.features ||
originalGeoJSON.features.length === 0) {
            alert("Les données des parcs ne sont pas encore complètement chargées.
Réessayez.");
            return;
        }

        // Call Turf buffering calculation algorithm routine passing target metrics
choices parameters
        var bufferedGeoJSON = turf.buffer(originalGeoJSON, distanceVal, { units:

```

```

'meters' });

    // Draw the computed overlay boundary envelope onto the map screen canvas
    bufferLayer = L.geoJSON(bufferedGeoJSON, {
        style: {
            color: '#2980b9',
            fillColor: '#3498db',
            weight: 2,
            opacity: 0.7,
            fillOpacity: 0.2
        }
    }).addTo(mymap);

    // Read buffer ring bounds matrix and smoothly frame camera viewport to
    encompass the entire overlay zone
    mymap.fitBounds(bufferLayer.getBounds());
    $(this).text("Supprimer le Buffer").removeClass("btn-warning").addClass("btn-
danger");
    } else {
        $(this).text("Générer le Buffer").removeClass("btn-danger").addClass("btn-
warning");
    }
    });

</html>

```